College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 6 Report: Filtering in the Frequency Domain – Laplacian and Sobel

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab applies edge-detection filters in the frequency domain instead of the spatial domain. The image is transformed with the 2-D FFT, multiplied by the frequency response of a Laplacian or Sobel filter, and transformed back with the inverse FFT. Everything was implemented in Python with NumPy, scikit-image and Matplotlib.

2. Image Used

The cameraman image is loaded from `skimage.data`. A copy is saved as `lab6/cameraman.png`:

- **cameraman.png** – 512 × 512 grayscale image of a man with a camera on a tripod, with a smooth sky, a textured grass field and strong edges along the coat, the tripod legs and the buildings.

3. Setup

The image is resized to 256 × 256 so the FFTs run faster. `resize` also converts it to floating point in the range [0, 1].

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from skimage.transform import resize

# Load cameraman image and resize for faster FFT
image = data.camera()
image = resize(image, (256, 256))

# Get image size
M, N = image.shape
```

4. Task 1 – Laplacian in the Frequency Domain

By the derivative property of the Fourier transform, taking the Laplacian of an image is the same as multiplying its spectrum by

$$H(u, v) = -4\pi^2 (u^2 + v^2)$$

`np.fft.fftfreq` gives the frequencies u and v in cycles per pixel, in the same unshifted order as `np.fft.fft2`, so the filter can be multiplied with the spectrum directly. The inverse FFT of the product

is the Laplacian of the image. Its imaginary part is only rounding error (about 10^{-15}), so the real part is kept.

```
# Generate the frequency grid
u = np.fft.fftfreq(M).reshape(-1, 1)
v = np.fft.fftfreq(N).reshape(1, -1)

# Compute the Laplacian filter in the frequency domain
laplacian_filter = -4 * (np.pi ** 2) * (u**2 + v**2)

# Apply the 2D FFT and the frequency domain filter
F = np.fft.fft2(image)
F_lap = F * laplacian_filter

# Inverse FFT to obtain the spatial result
laplacian_image = np.fft.ifft2(F_lap).real

# Display the original image, filtered spectrum, and spatial output
fig, ax = plt.subplots(1, 3, figsize=(12, 5))
ax[0].imshow(image, cmap="gray", vmin=0, vmax=1)
ax[0].set_title("Original Cameraman")
ax[0].axis("off")

laplacian_spectrum = np.log1p(np.abs(np.fft.fftshift(F_lap)))
ax[1].imshow(laplacian_spectrum, cmap="gray")
ax[1].set_title("Laplacian Spectrum")
ax[1].axis("off")

display_limit = np.max(np.abs(laplacian_image))
ax[2].imshow(laplacian_image, cmap="gray", vmin=-display_limit, vmax=display_limit)
ax[2].set_title("Laplacian (Spatial Output)")
ax[2].axis("off")

plt.tight_layout()
plt.show()
```

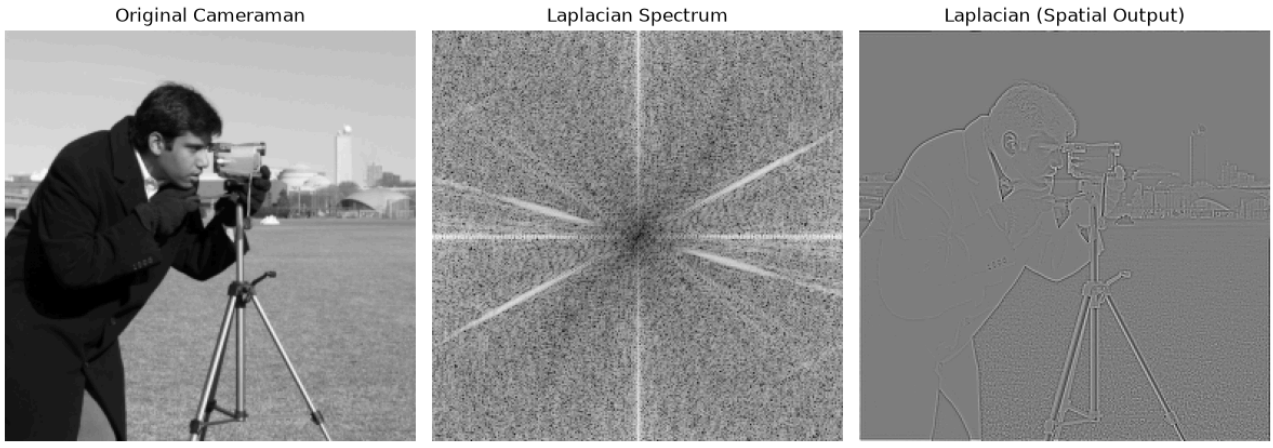


Figure 1: Original image, log-magnitude of the filtered spectrum (centred with fftshift) and the Laplacian in the spatial domain.

Discussion: H(u, v) is zero at the centre (DC) and grows with the square of the distance from it, up to about −19.7 at the highest frequencies. The filter is therefore a high-pass filter: in the filtered spectrum

the centre is dark because the low frequencies are suppressed, while the outer high frequencies keep a lot of energy. The bright diagonal streaks come from the long straight edges in the image, such as the tripod legs, and the horizontal and vertical lines through the centre come from the image borders. Because the DC value is removed, the spatial output has a mean of zero, so it is shown with a symmetric range where mid-gray means zero. Flat regions like the sky are mid-gray, and every edge appears as a pair of bright and dark lines, which are the zero crossings of the second derivative. The grass texture also shows up clearly because the filter boosts fine detail and noise. The result is very close to the spatial 3×3 Laplacian (correlation ≈ 0.97). The FFT treats the image as periodic, so the top and bottom edges and the left and right edges are treated as neighbours. Their brightness does not match, which creates an artificial edge around the border: the mean absolute response there is about 0.31, compared with 0.09 inside the image.

5. Assessment Task 1 – Sobel Filters in the Frequency Domain

The 3×3 Sobel kernels from the lab manual are applied with the FFT. A small kernel must first be zero-padded to the size of the image so that the two spectra can be multiplied. The helper `center_embed_kernel` places the kernel at the centre of a 256 × 256 array of zeros and then uses `np.fft.ifftshift` to move the kernel's centre to pixel (0, 0). Without this shift, the output would be displaced by half of the image size. The FFT of this array is the filter's frequency response.

```
# Sobel filters from the lab manual
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]])

sobel_y = np.array([[-1, -2, -1],
                    [ 0,  0,  0],
                    [ 1,  2,  1]])

# Center a spatial filter in the image-sized frequency grid
def center_embed_kernel(kernel, shape):
    padded = np.zeros(shape)
    kh, kw = kernel.shape
    ph, pw = shape

    # Find center coordinates
    cy, cx = ph // 2, pw // 2

    # Insert the kernel at the center, then shift it to the origin
    padded[cy - kh // 2:cy - kh // 2 + kh,
           cx - kw // 2:cx - kw // 2 + kw] = kernel

    return np.fft.fft2(np.fft.ifftshift(padded))
```

Both filters are multiplied with the image spectrum, and the gradient magnitude is computed from the two results as $\sqrt{(G_x^2 + G_y^2)}$ with `np.hypot`.

```
# Compute the FFT of the image and centered Sobel filters
F = np.fft.fft2(image)
sobel_x_frequency = center_embed_kernel(sobel_x, image.shape)
sobel_y_frequency = center_embed_kernel(sobel_y, image.shape)

# Apply both Sobel filters in the frequency domain
sobel_x_image = np.fft.ifft2(F * sobel_x_frequency).real
sobel_y_image = np.fft.ifft2(F * sobel_y_frequency).real
sobel_magnitude = np.hypot(sobel_x_image, sobel_y_image)

fig, ax = plt.subplots(1, 4, figsize=(15, 4))
ax[0].imshow(image, cmap="gray", vmin=0, vmax=1)
ax[0].set_title("Original Cameraman")
ax[0].axis("off")

ax[1].imshow(sobel_x_image, cmap="gray")
ax[1].set_title("Sobel X (Frequency Domain)")
ax[1].axis("off")

ax[2].imshow(sobel_y_image, cmap="gray")
ax[2].set_title("Sobel Y (Frequency Domain)")
ax[2].axis("off")

ax[3].imshow(sobel_magnitude, cmap="gray")
ax[3].set_title("Sobel Gradient Magnitude")
ax[3].axis("off")

plt.tight_layout()
plt.show()
```

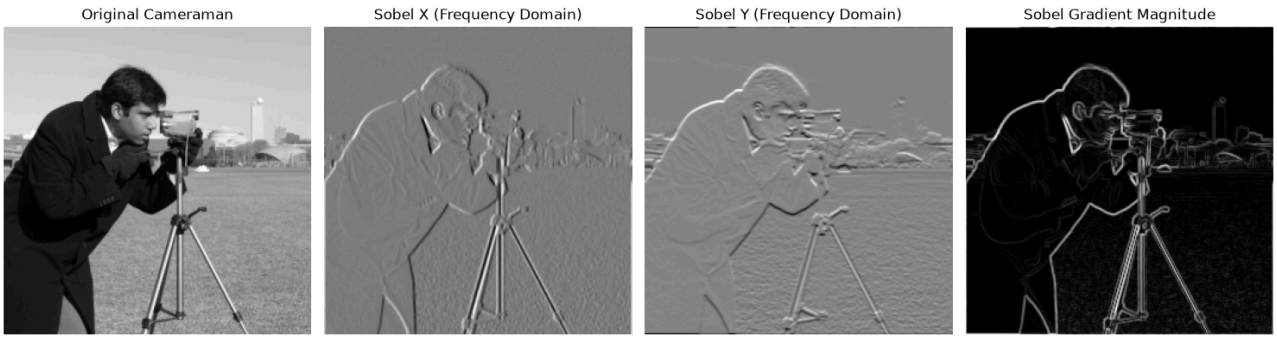


Figure 2: Original image, Sobel X and Sobel Y responses computed with the FFT, and the gradient magnitude.

Discussion: The Sobel X result responds to changes in the horizontal direction, so vertical edges such as the tripod legs, the building towers and the side of the coat stand out, while horizontal edges almost disappear. Sobel Y does the opposite and highlights horizontal edges such as the horizon, the camera and the shoulders. Both images have an embossed look because the sign of the response shows whether the intensity rises or falls across the edge. The kernels' weights sum to zero, so the flat sky gives almost no response: the mean absolute Sobel X value is about 0.03 in the sky and 0.22 in the grass. The gradient magnitude combines both directions, giving a clean edge map in which the outline of the cameraman, the tripod and the buildings are bright on a dark background. Multiplying spectra corresponds to *convolution*, not correlation, so the kernel is effectively flipped and the signs of

G_x and G_y are reversed compared with applying the kernel by correlation (for example with `cv2.filter2D`). The magnitude is not affected by this. The result matches spatial convolution with wrap-around borders to within 10^{-15}, which confirms that the padding and centring are correct. As in Task 1, the periodic FFT causes a bright frame at the image border, where the mean magnitude is about 1.15 compared with 0.22 inside. Zero-padding the image before the FFT would remove this artefact.

6. Conclusion

Linear filters can be applied in the frequency domain by multiplying the image spectrum with the filter's frequency response, because convolution in the spatial domain becomes multiplication after the Fourier transform. The Laplacian can be written directly as the formula $-4\pi^2(u^2 + v^2)$, which shows clearly that it is a high-pass filter. A spatial kernel such as Sobel needs to be zero-padded to the image size and centred at the origin before its FFT is taken. Both approaches give the same edges as spatial filtering. The main difference is that the FFT assumes the image repeats periodically, which creates false edges at the borders unless the image is padded. For small 3×3 kernels spatial filtering is simpler, but the frequency-domain method becomes faster for large kernels and makes it easy to see which frequencies a filter keeps or removes.

7. References

Sandipan Dey, *Hands-On Image Processing with Python*, Packt.

NumPy: Discrete Fourier Transform (numpy.fft) – numpy.org/doc/stable/reference/routines.fft.html

OpenCV: Fourier Transform – docs.opencv.org/4.x/de/dbc/tutorial_py_fourier_transform.html